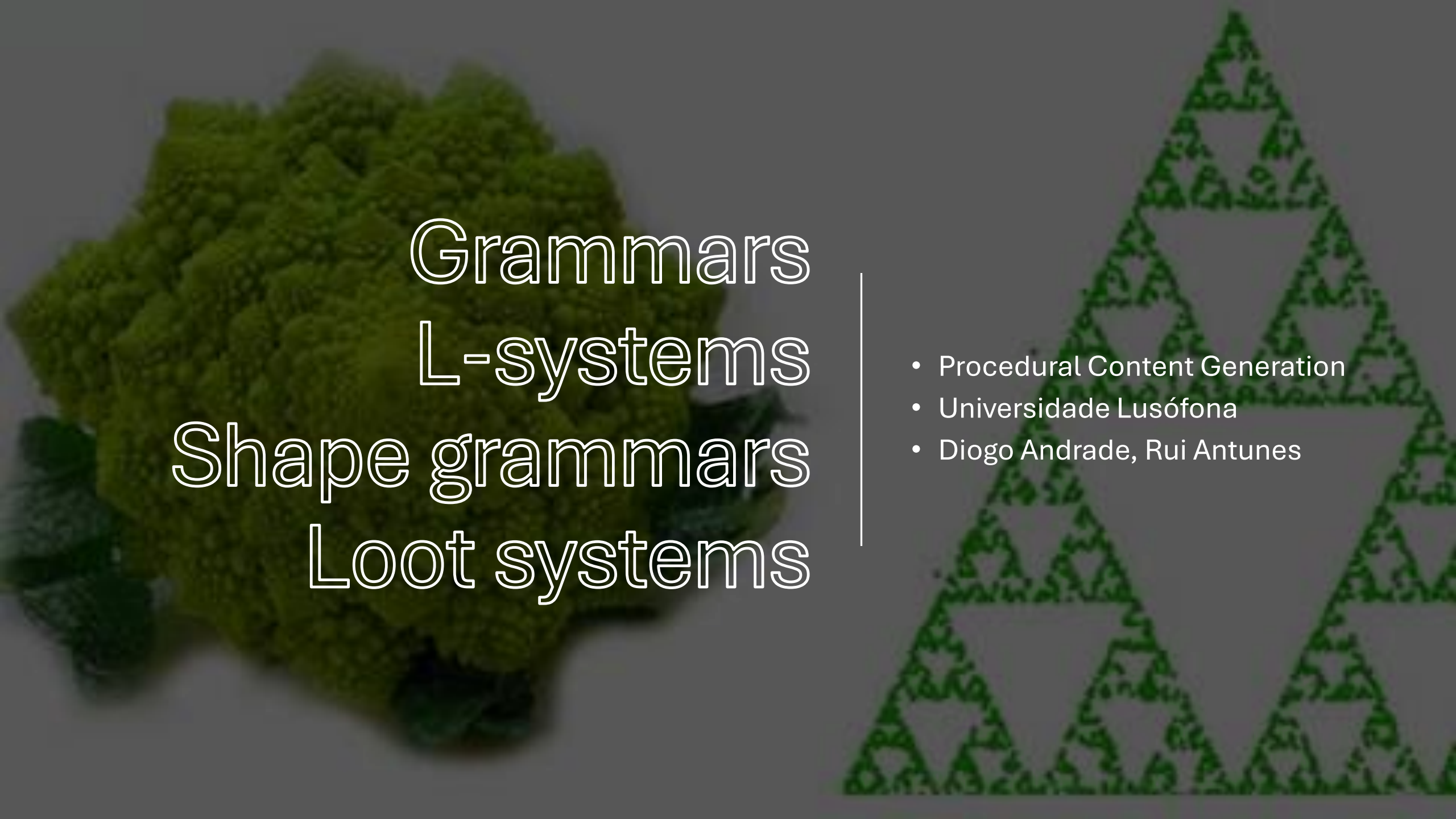
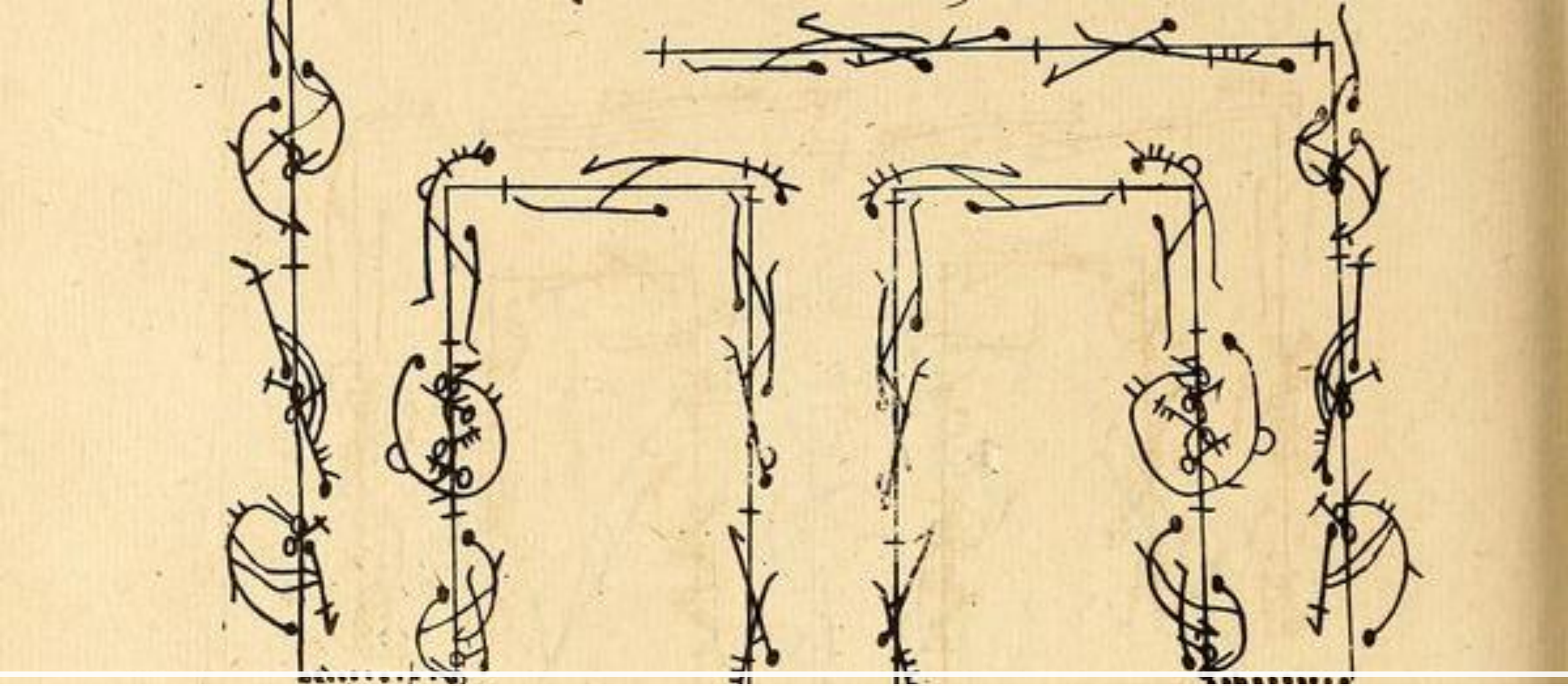
Grammars
L-systems
Shape grammars
Loot systems



- Procedural Content Generation
- Universidade Lusófona
- Diogo Andrade, Rui Antunes

- What is a grammar?
- Grammars as generative systems
- Deterministic vs non-deterministic
- L-systems (biology)
- Shape grammars (architecture)
- Procedural generation (games)
- Loot systems (stochastic grammars)
- Narrative grammars



Formal Grammars

Simple sentence grammar

Formal grammars were originally introduced in 1950 by the linguist *Noam Chomsky* as the way to model natural language.

Symbols

Non-terminal symbols (placeholders):

- S – Sentence
- NP – Noun Phrase
- VP – Verb Phrase
- Det – Determiner
- N – Noun
- V – Verb

Terminal symbols (actual words):

- Determiners: the, a
- Nouns: cat, dog
- Verbs: chases, sleeps

Production Rules

S → NP VP
NP → Det N
VP → V NP | V
Det → the | a
N → cat | dog
V → chases | sleeps

Starting Symbol

Start: S

S
⇒ **NP VP**

(NP → Det N)
⇒ **Det N VP**

(VP → V NP)
⇒ **Det N V NP**

Det → the
N → cat
V → chases
Det → a
N → dog

The cat chases a dog

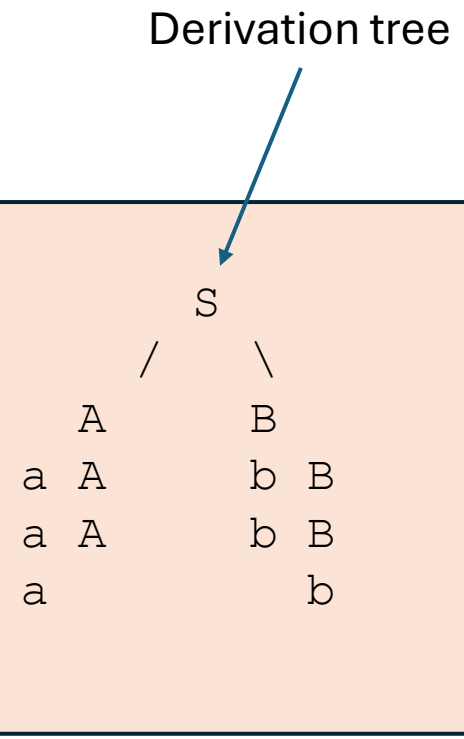
Formal grammars

- Alphabet (symbols)
- Production rules
- Start symbol (Axiom)
- Derivation tree
- Language generated by the grammar

Example:
Production rules:
S → AB
A → aA | a
B → bB | b

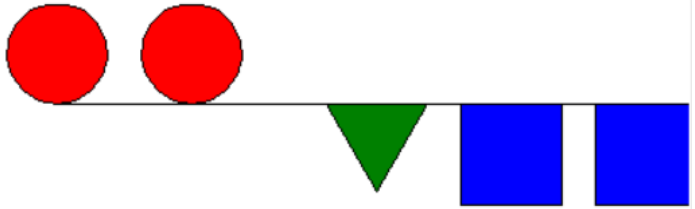
Category	Symbols
Terminals	a, b
Non-terminals	S, A, B
Axiom	S

```
S
⇒ AB
⇒ aA B
⇒ aaA B
⇒ aaa B
⇒ aaa bB
⇒ aaa bbB
⇒ aaa bbb
```



Artists can define a “**visual grammar**” to generate or analyze artwork.

Example:
A rule could produce: Red circle next to blue square, then expand recursively to create complex patterns or generative art.



Symbols:

- Basic shapes: circle, square, triangle
- Colors: red, blue, green
- Layout elements: line, curve

Production rules:

- Artwork → Element Artwork | Element
- Element → Shape | Shape + Color | Group(Element, Element)

Musical Grammar

In music, a “grammar” can define how notes, chords, and rhythms are combined.

Symbols:

- Notes: C, D, E, F, G, A, B
- Chords: Cmaj, G7, Am
- Rhythm patterns: quarter, half, whole

Production rules:

- Melody → Note Melody | Note
- ChordProgression → Chord ChordProgression | Chord

Example:
Start symbol: Song
Rules generate a sequence like C D E F G A B or chord progression Cmaj Am G7 Cmaj.

This approach is used in **algorithmic composition**, like in the works of Iannis Xenakis.

Exercise 1

1.1

Grammar

S → AB

A → aA | a

B → bB | b

Tasks:

1. Generate **5 valid strings** from the grammar.
2. Draw the **derivation tree** for one string.

1.2

Create a grammar that generates

arithmetic expressions such as
3+4, (2+5)*7, 8*(1+3)

1.3

Create a grammar to generate **fantasy sentences**.

Example output:

The wizard escapes from a dragon
The rogue befriends an orc

Simple dungeon generator

Axiom:

S

Rules:
S → Room Corridor Room
Room → TreasureRoom | MonsterRoom | EmptyRoom
Corridor → Straight | Turn

Generated result might be:
TreasureRoom – Straight – MonsterRoom

Deterministic vs **Non-Deterministic** Grammars

Deterministic grammar

- At each step, **exactly one rule** that applies to each symbol or sequence of symbols
- Completely unambiguous

Example:

A → AB
B → A

Nondeterministic grammars

- **Multiple choices** possible, since several rules could apply to a given string
- The grammar might even include probabilities for choosing each rule
- another way is to use parameters for deciding which way to expand the grammar.

Example:

A → AB | BA | AA

Key idea:

Non-determinism introduces **variation and emergence**.

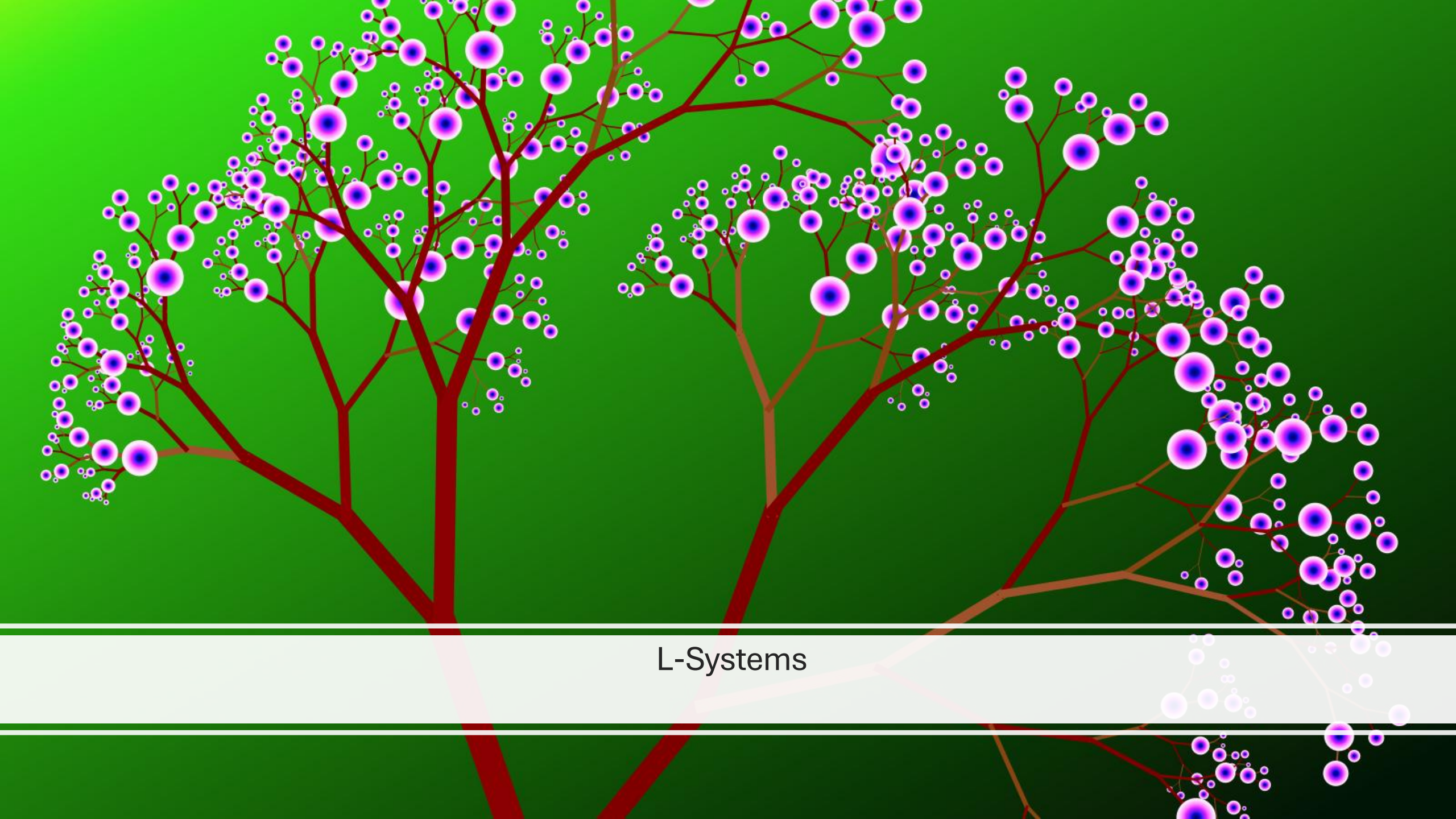
Sequential vs **Paralell** (re)writing

Sequential writing

- goes through the string from left to right and rewrites the string **as it is reading it**
- If a production rule is applied to a symbol the result of that rule is written into the very same string before the next symbol is considered

Parallel rewriting

- **all** the rewriting is done **at the same time**
- this is implemented as the insertion of a new string at the separate memory location containing all the efects of applying the rules, while the original string is left unchanged.



L-Systems

L-Systems

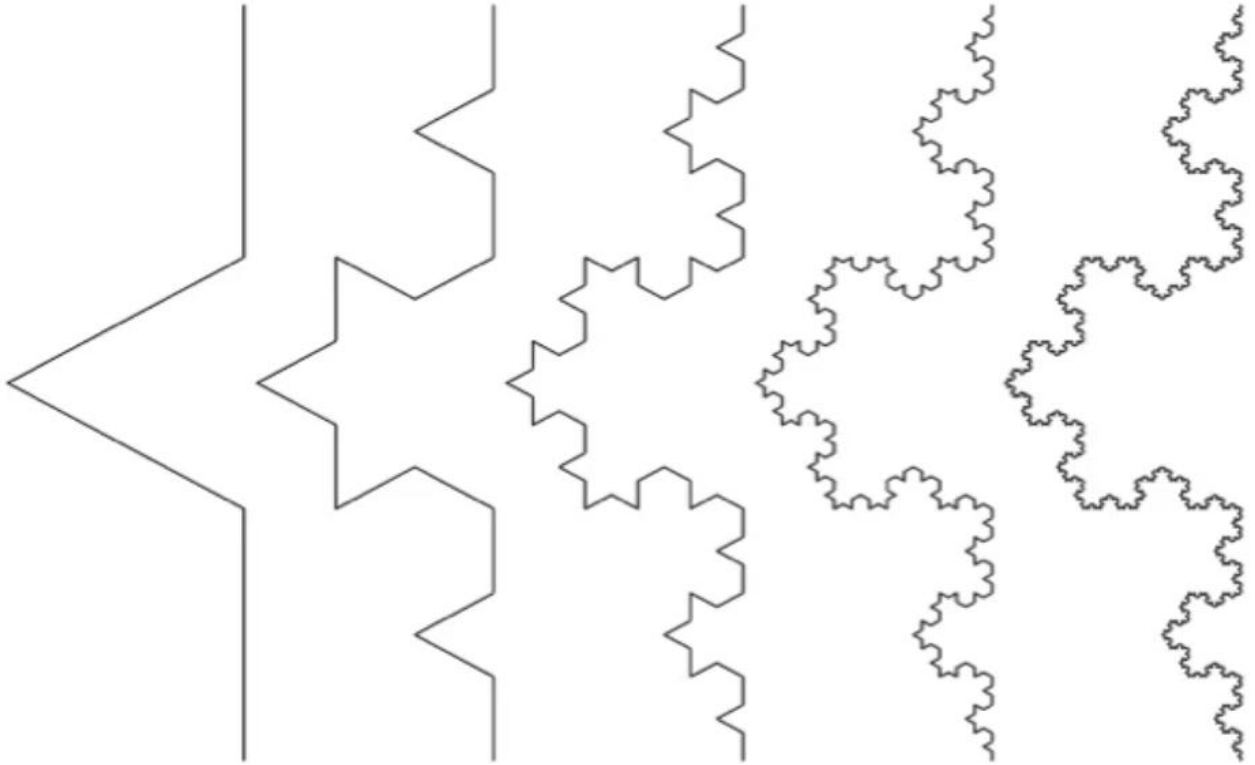
- Class of grammars whose defining feature is **parallel rewriting**.
- Biologist *Prusinkiewicz and Lindenmayer* in 1968 (Model yeast growth)
- Model the growth of organic systems such as plants and algae

Example
A→ AB
B→ A

A
AB
ABA
ABAAB
ABAABABA
ABAABABAABAAB
ABAABABAABAABABAABABA
ABAABABAABAABABAABABAABAABABAABAAB

Fibonacci 1,2,3,5,8,13,21,34,55,89
Outcome of N=n+ (n-1) + (n-2)

KOCH CURVE

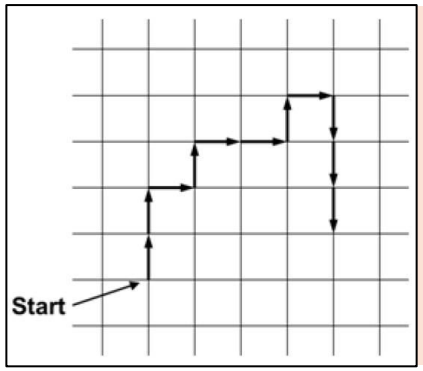


Niels Fabien Helge von Koch, 1906

The Koch curve rules:

- **Axiom:** F
- **Rule:** F → F+F−F-F +F
- **Angle:** 60°

F = move forward (a certain distance) and draw
+ = turn left
- = turn right



Turtle graphics

Think of the turtle as moving across a plane **holding a pencil** and simply drawing **a line that traces its path**

interpreting all systems as a turtle allows us to enter into the **fractal dimension**

```
import turtle
def lsystem(axiom, rules, iterations):
    current = axiom
    for i in range(iterations):
        next_string = ""
        for char in current:
            next_string += rules.get(char, char)
        current = next_string
    return current
axiom = "F"
rules = {
    "F": "F+F-F-F+F"
}

LSystem_result = lsystem(axiom, rules, 4)
```

```
def draw_lsystem(commands, angle=90, step=6):
    for c in commands:
        if c == "F":
            turtle.forward(step)
        elif c == "+":
            turtle.right(angle)
        elif c == "-":
            turtle.left(angle)

commands = LSystem_result
turtle.speed(0)
turtle.penup()
turtle.goto(-250, 150)
turtle.pendown()
draw_lsystem(commands)
turtle.done()
```

Exercise 2 - Create different patterns
1-Change the axiom
2-Change rules.

Bracketed L systems

have two extra symbols **[** and **]**

They act as **push** and **pop** commands to a stack (FILO)

[- **Saves** the current position and orientation of the turtle onto the stack

] - **Retrieves** the last saved position from the stack and reset the turtle to that position → The turtle jumps back to a position it has previously been at

F→F[-F]F[+F][F]

30º

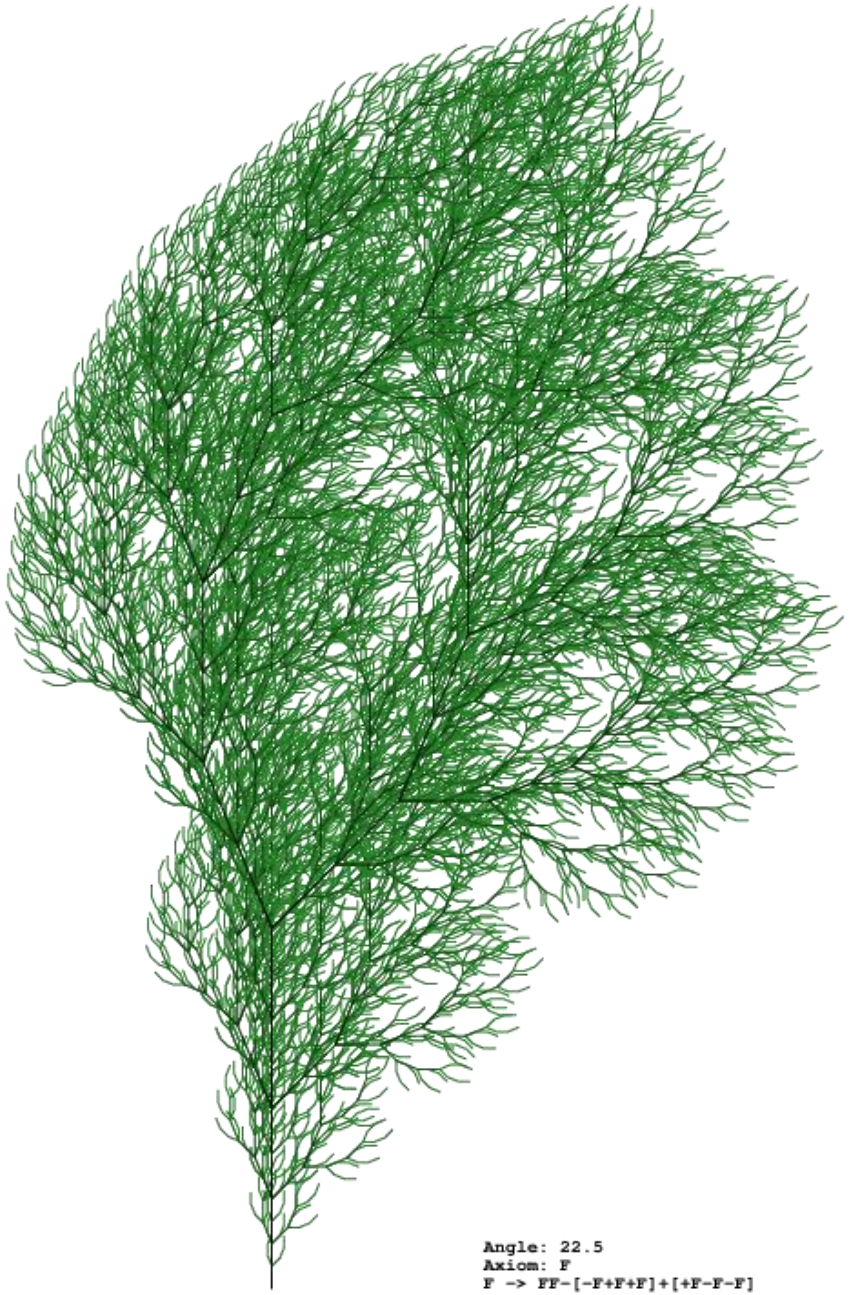
Plant Generation Example

Classic rule set:

F → F[-F]F[+F][F]

Example in the picture:

F→FF-[-F+F+F]+[+F-F-F]



- Exercícios

Given:

Axiom: F

Rules: F → F+F−F−F+F

Angle: 90°

Tasks:

1. Compute **iteration 1**
2. Compute **iteration 2**
3. Sketch the resulting pattern

Given:

Axiom: X

Rules:

X → F+[[X]-X]-F[-FX]+X

F → FF

Tasks:

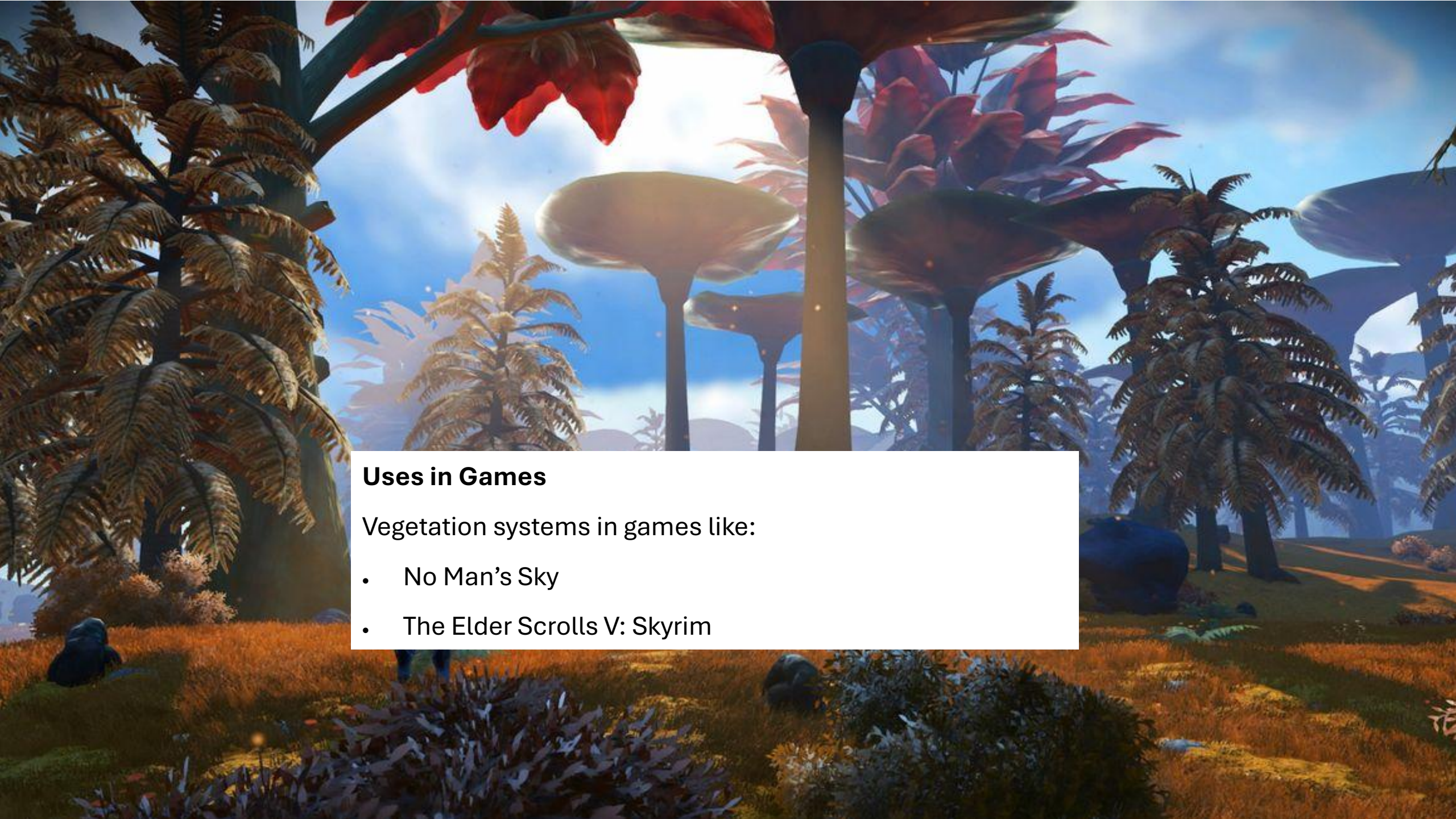
1. Expand **2 iterations**
2. Sketch the plant structure.

```
import pygame
import math


 pygame.init()
WIDTH, HEIGHT = 1000, 800
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("L-System Tree")
clock = pygame.time.Clock()


# L-system parameters
axiom = "F"
rule = {"F": "F[-F]F[+F][F]"}
angle = 30
iterations = 0
```

```
def draw_lsystem(sequence, length):
    x = WIDTH // 2
    y = HEIGHT - 50
    heading = -90  # start pointing up
    stack = []
    for command in sequence:
        if command == "F":
            new_x = x + length * math.cos(math.radians(heading))
            new_y = y + length * math.sin(math.radians(heading))
            pygame.draw.line(screen, (0, 255, 100), (x, y), (new_x, new_y), 1)
            x, y = new_x, new_y
        elif command == "+":
            heading += angle
        elif command == "-":
            heading -= angle
        elif command == "[":
            stack.append((x, y, heading))
        elif command == "]":
            x, y, heading = stack.pop()
```

Exercise 3
Open Lsystem_fern.py
Implement the following rule

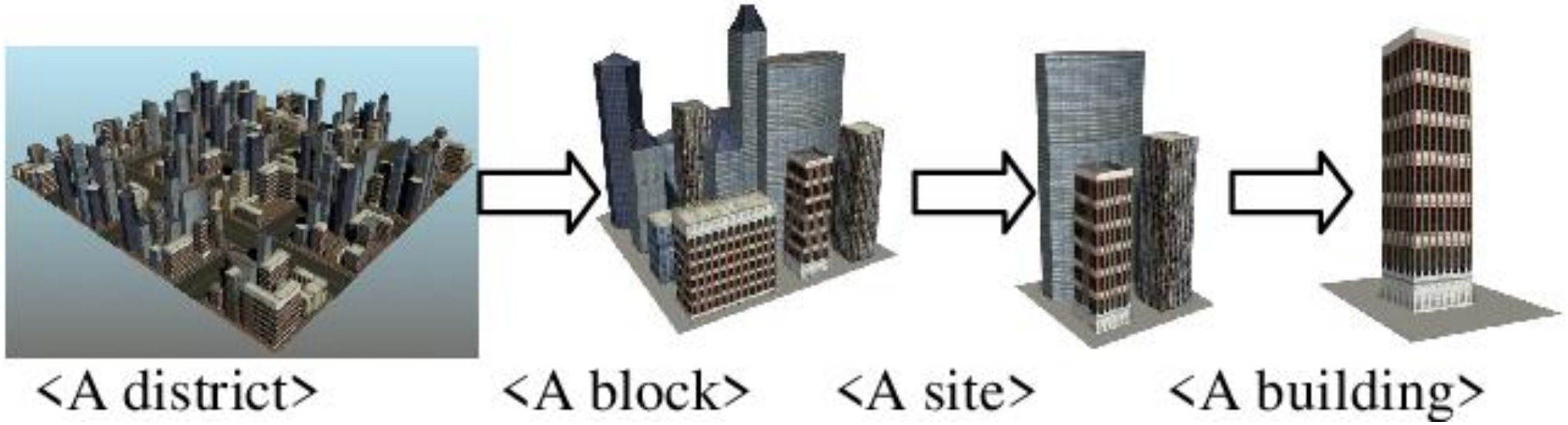
F→FF-[-F+F+F]+[+F-F-F]



Uses in Games

Vegetation systems in games like:

- No Man’s Sky
- The Elder Scrolls V: Skyrim



Shape Grammars

Shape grammars are similar to formal grammars but operate on **visual shapes instead of symbols**.

They were originally introduced by *George Stiny* and *James Gips* for architectural design.

Instead of:

S → A B

you have rules like:

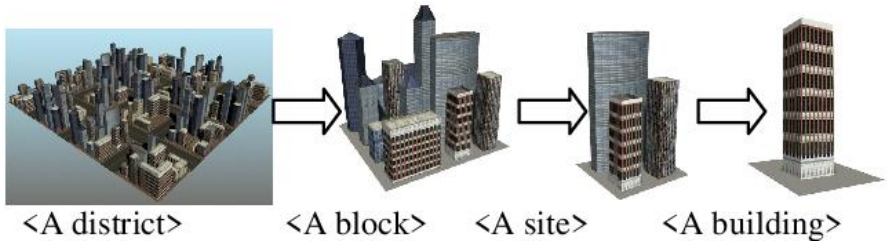
Square → Square + Door
Square → Square + Window
Square → Two smaller Squares

These rules **transform geometric shapes**.

Why Shape Grammars Are Useful

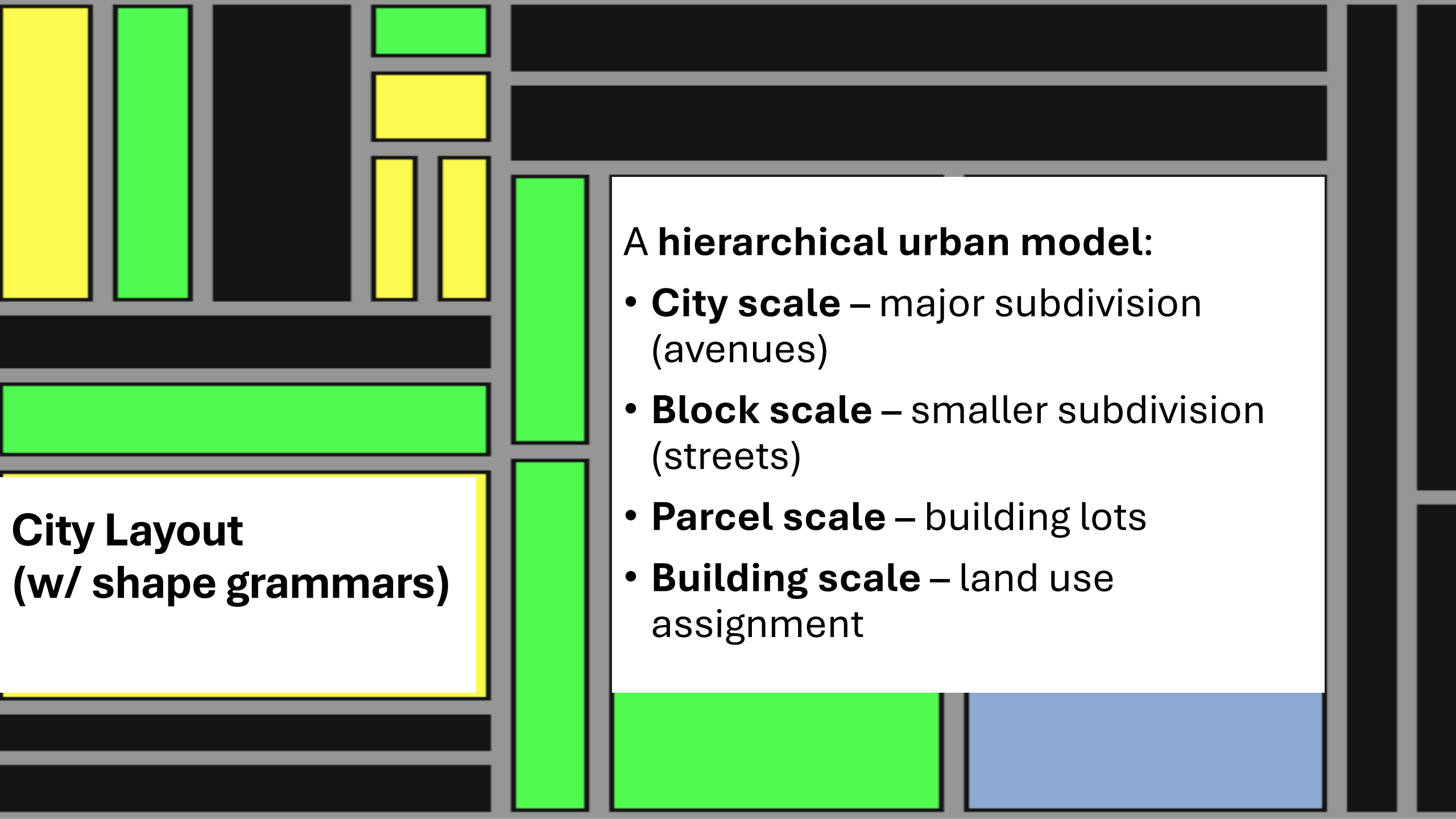
They allow:

- architectural style control
- consistent building patterns
- scalable cities



Example pipeline:

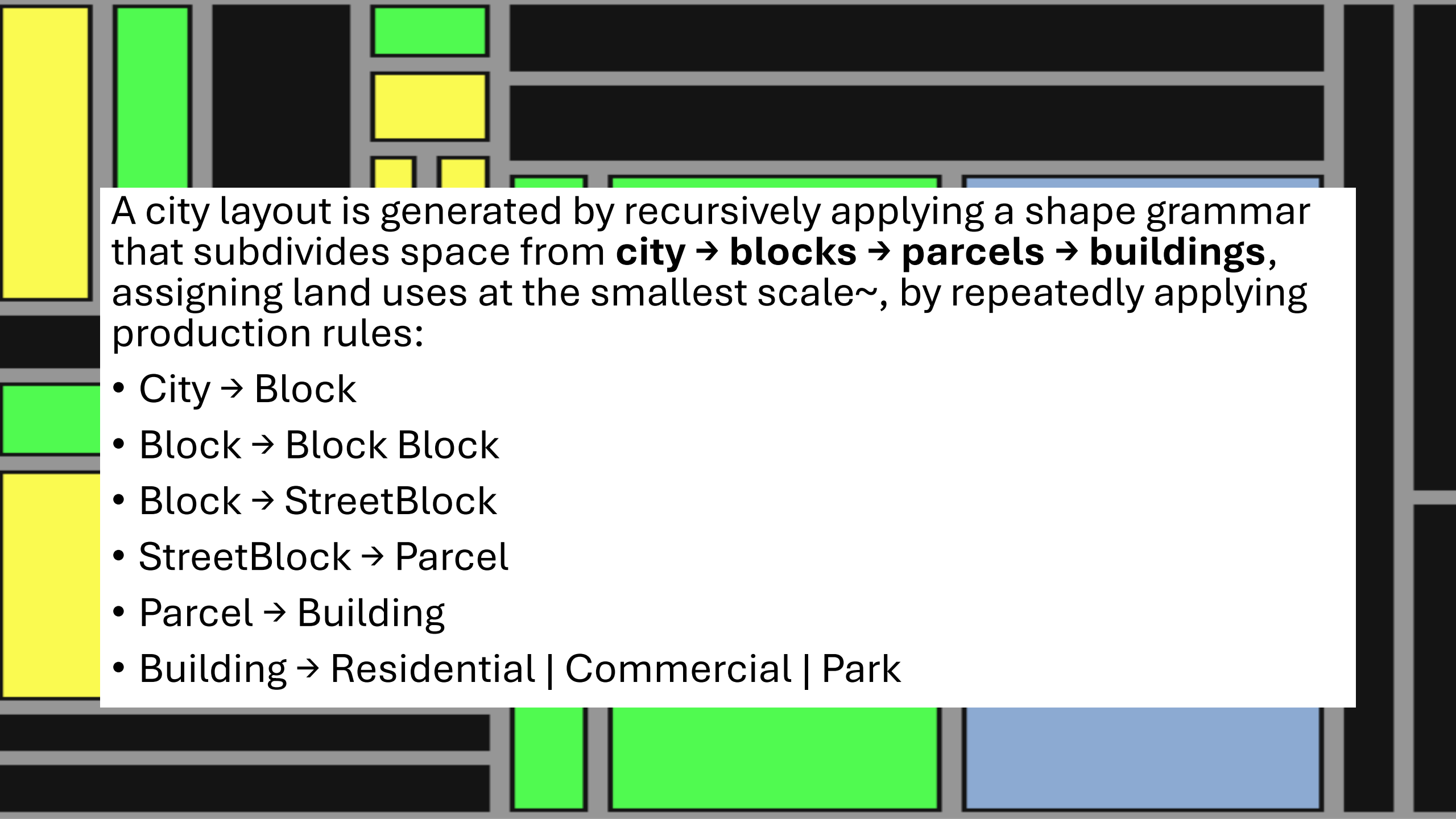
city layout → block → building → windows → decorations



City Layout (w/ shape grammars)

A **hierarchical urban model**:

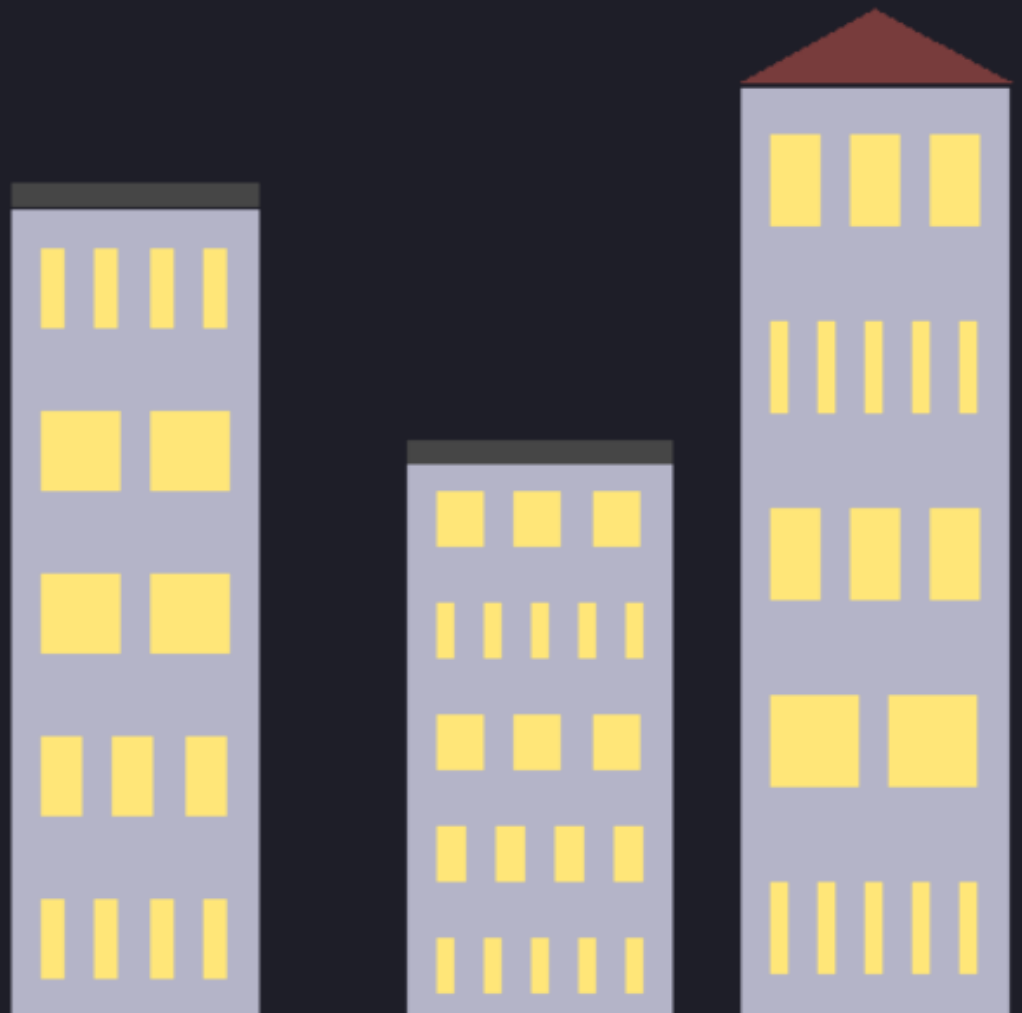
- **City scale** – major subdivision (avenues)
- **Block scale** – smaller subdivision (streets)
- **Parcel scale** – building lots
- **Building scale** – land use assignment

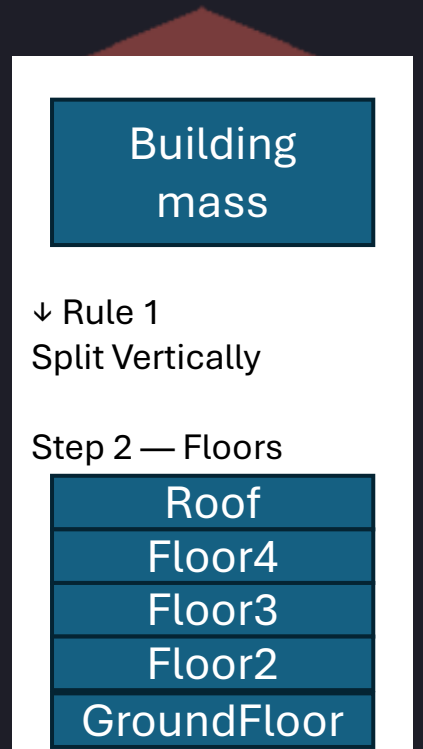


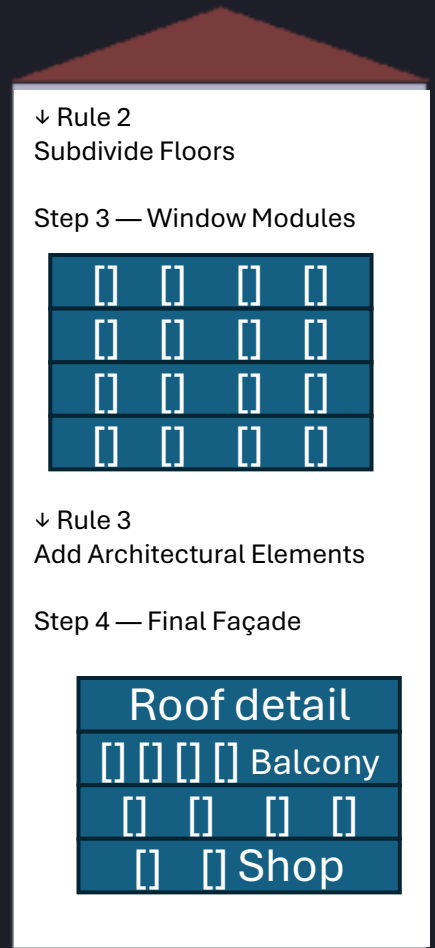
A city layout is generated by recursively applying a shape grammar that subdivides space from **city → blocks → parcels → buildings**, assigning land uses at the smallest scale~, by repeatedly applying production rules:

- City → Block
- Block → Block Block
- Block → StreetBlock
- StreetBlock → Parcel
- Parcel → Building
- Building → Residential | Commercial | Park

Procedural Building Generation







Games using similar techniques include Assassin's Creed Unity and SimCity.

Freiknecht, J.; Effelsberg, W. A Survey on the Procedural Generation of Virtual Worlds. *Multimodal Technol. Interact.* **2017**, *1*, 27. https://doi.org/10.3390/mti1040027

Exercises

Square Subdivision

Start shape:

□

Rules:

1. Split square horizontally
2. Split square vertically
3. Replace square with 4 smaller squares

Tasks:

- Apply **3 steps**
- Draw the resulting pattern.

House Grammar

Rules:

House → Base + Roof
Base → rectangle
Roof → triangle

Additional rules:

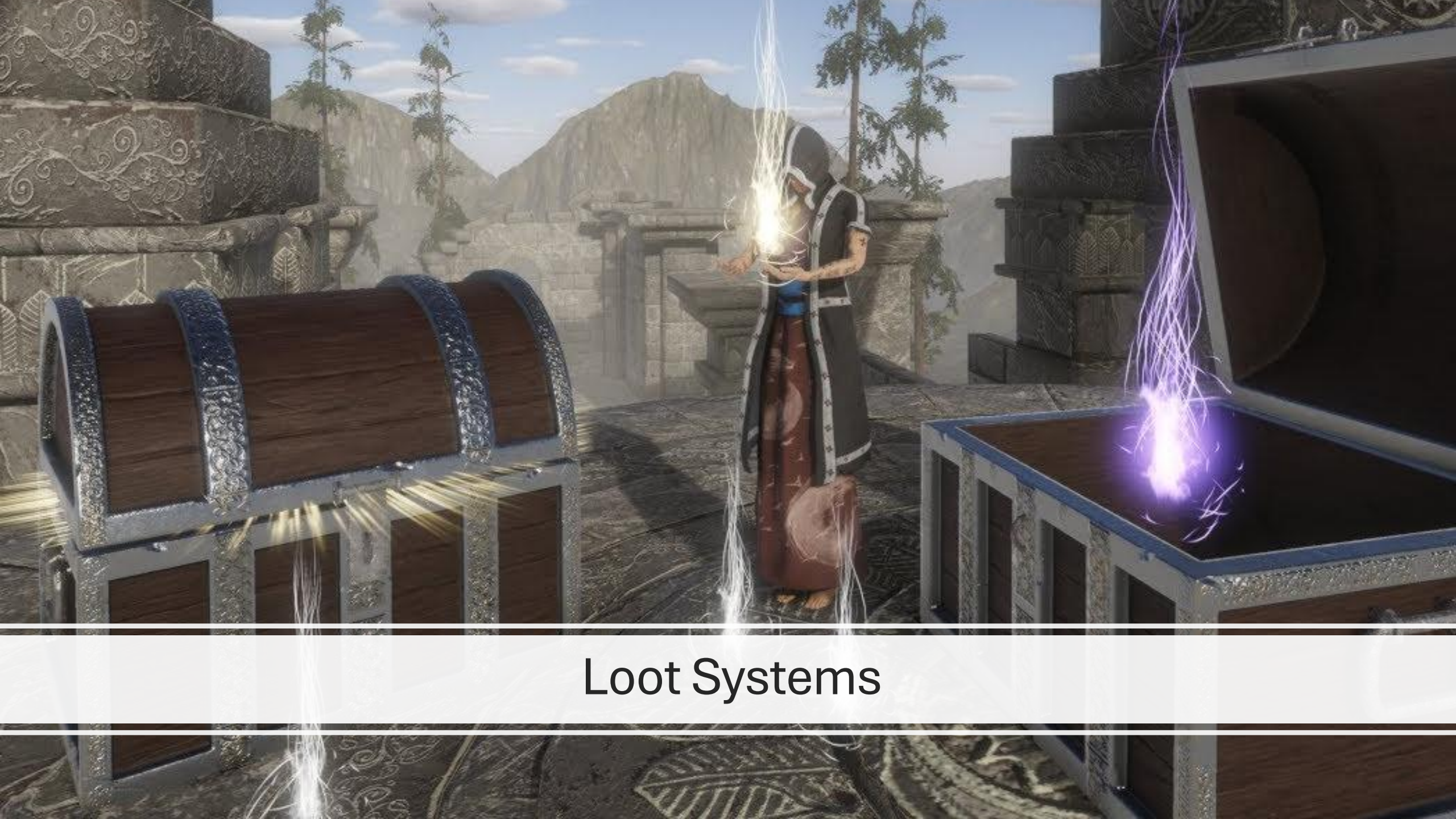
Base → Base + Window
Base → Base + Door

Task:

- Generate **3 different houses** using the rules.

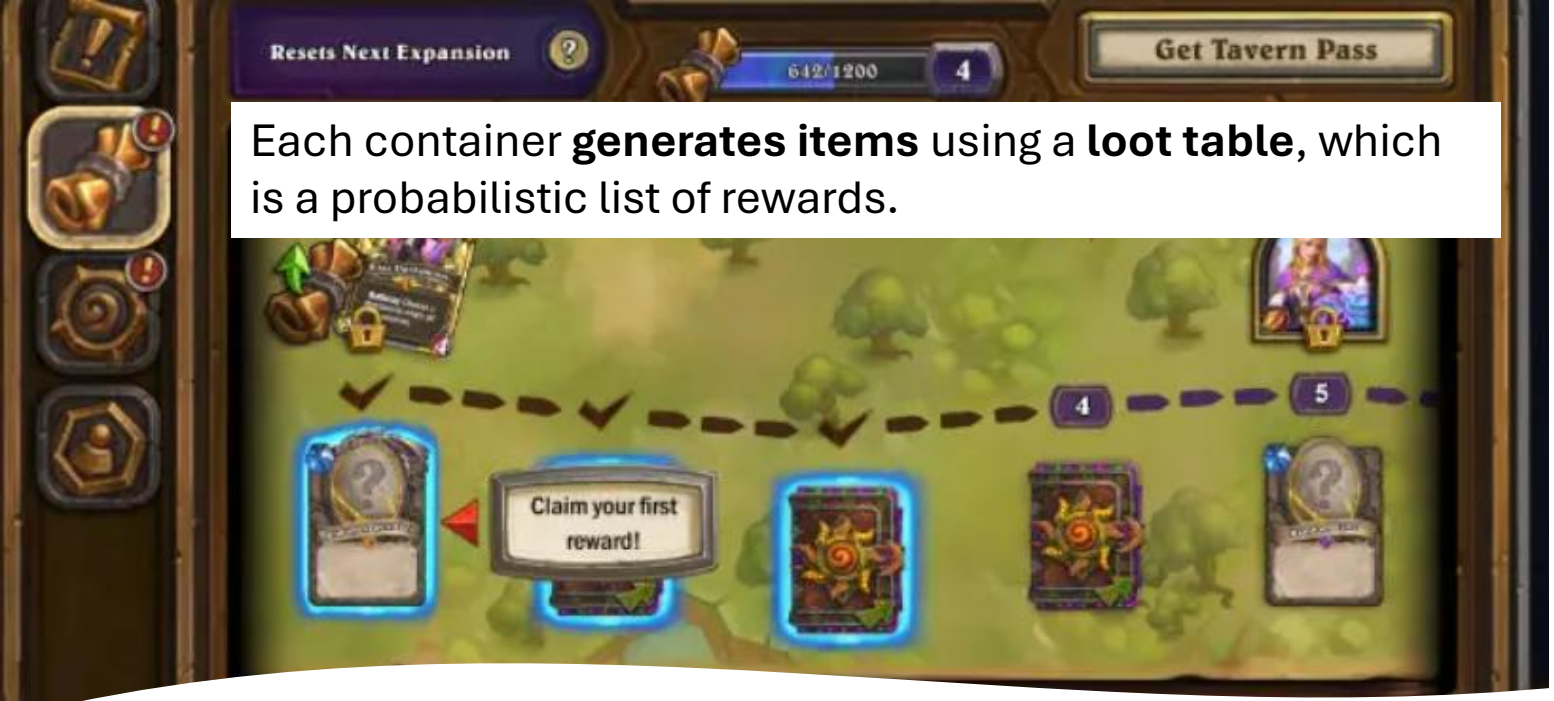
Open Shape_grammar_city_layout_explicit.py

And extend the grammar and the implementation to include commercial zones



Loot Systems

- A **loot system** is the mechanism for generating, dropping, and **distributing items** (such as **weapons, armor,** or **currency**)
- Players collect these item to increase their power, customize characters, or progress.
- It acts as a reward loop, often utilizing rarity tiers and randomization (random number generators) to provide excitement and engagement
- loot often forms the core economy of the game, in which the player fights to obtain loot and then uses it to purchase other items
- The concept of color-coded loot rarity was initially popularized with the 1996 game *Diablo* and its 2000 sequel *Diablo II*



Possible attributes:
Sword: +15–25 damage
Sword: +5% critical chance
Sword: +10% fire damage
Sword: +8% ice damage

LootTable

A loot table defines **probabilities of item types**.
Gold → 40%
Potion → 30%
Sword → 20%
RareItem → 10%

Weapons may receive **random modifiers**.
Sword → BaseSword + Modifier*

Modifier →
+Damage
+CriticalChance
+ElementalDamage

Monster kill is a mechanic where defeated enemies directly provide loot, materials, or equipment, often used to craft or upgrade player gear.

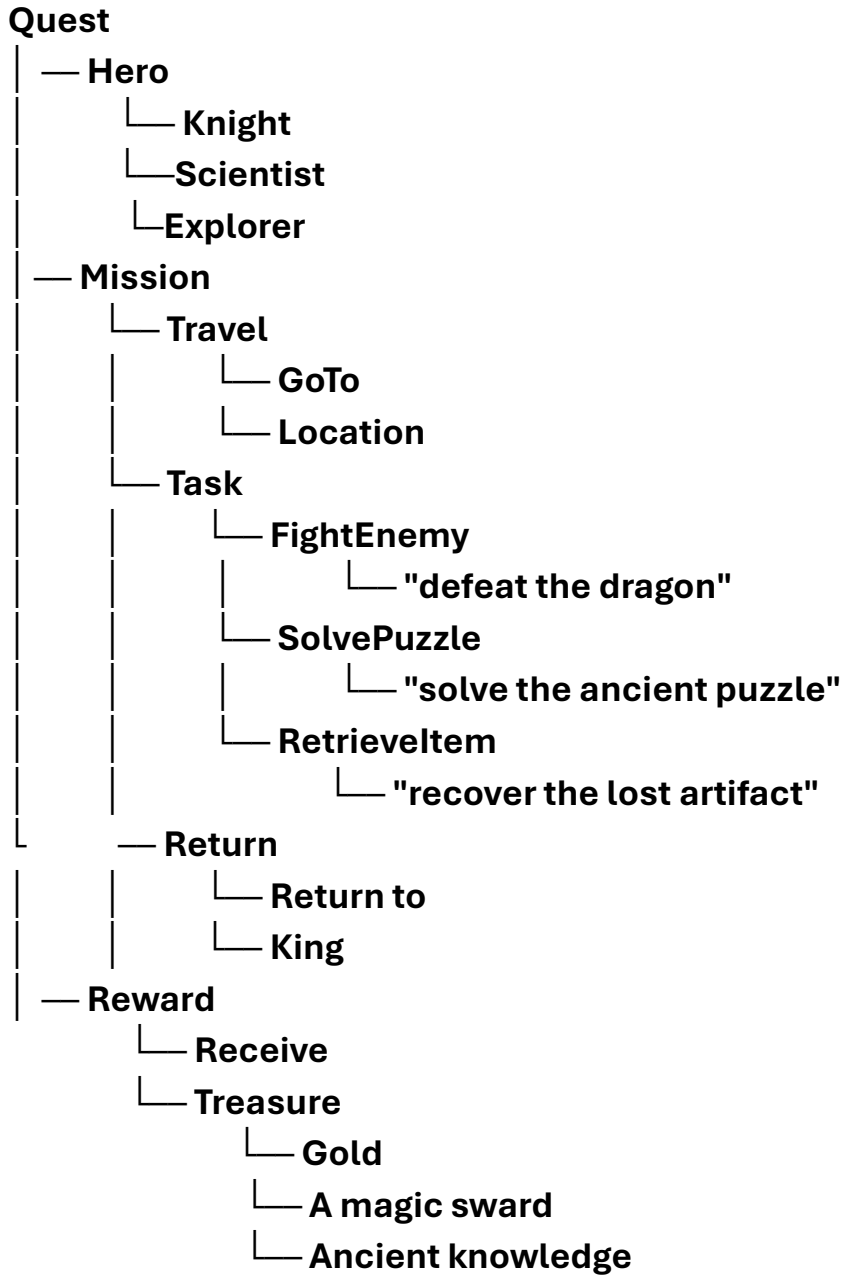
This system promotes direct progression—killing specific monsters to harvest their unique parts (e.g., teeth, scales) to craft better items.

- **Direct Harvesting (Carving):** Players often "carve" or skin fallen monsters immediately to gather parts, as seen in Monster Hunter: World.
- **Breakable Parts:** Damaging specific monster parts (e.g., breaking wings or cutting tails) can yield special, rare drops.
- **Crafting Focus:** Instead of just finding a "better sword," you find the monster parts to craft that sword.
- **"You Keep What You Kill":** Common in ARPGs (e.g., *Borderlands*, *Grim Dawn*), where enemies drop the exact weapons or gear they were using against you.
- **Drop Tables/Rarity:** Loot is often categorized into breakable parts, carve rewards, or quest rewards, each with different probabilities, often managed by a "field guide" or loot table,


Quest List
Example Quest text
Objectives:
Go to the town hall and agrravate the guard, and eat dinner
Go to the town hall and agrravate the guard
Go to the town hall and agrravate the guard
Rewards:
300 coins, 1 healthy hat
Example Quest 02/50
Example Quest 02/50
Example Quest 02/50
Example Quest 02/50
Example Quest 02/50

Quests

```python
import random

grammar = {
    "Quest": [["Hero", "Mission", "Reward"]],
    "Hero": [["Knight"], ["Scientist"], ["Explorer"]],
    "Mission": [["Travel", "Task", "Return"]],
    "Travel": [["Go to", "Location"]],
    "Task": [["FightEnemy"], ["SolvePuzzle"], ["RetrieveItem"]],
    "Return": [["Return to", "King"]],
    "Reward": [["Receive", "Treasure"]],
    "Location": [["the forest"], ["the ruins"], ["the cave"]],
    "FightEnemy": [["defeat the dragon"]],
    "SolvePuzzle": [["solve the ancient puzzle"]],
    "RetrieveItem": [["recover the lost artifact"]],

    "Treasure": [["gold"], ["a magic sword"], ["ancient knowledge"]]
}

def expand(symbol):
    if symbol not in grammar:
        return symbol
    rule = random.choice(grammar[symbol])
    return " ".join(expand(s) for s in rule)
print(expand("Quest"))
```



Exercises

Context-Free Grammar (CFG)

#1. Add locations (forest, dungeon, castle)

#2. Add quest rewards

#3. Generate 5 different quests

#4. add **conditions** (e.g., only certain actions for certain enemies **Context Sensivity**)

S → Hero must Action the Enemy
Hero → knight | wizard | rogue | ranger
Action → defeat | rescue | find | protect
Enemy → dragon | necromancer | goblin king | giant spider

```
import random

grammar = {
    "S": [["Hero", "must", "Action", "the", "Enemy"]],
    "Hero": [["knight"], ["wizard"], ["rogue"], ["ranger"]],
    "Action": [["defeat"], ["rescue"], ["find"], ["protect"]],
    "Enemy": [["dragon"], ["necromancer"], ["goblin king"], ["giant spider"]]
}
def generate(symbol):
    if symbol not in grammar:
        return symbol
    rule = random.choice(grammar[symbol])
    return " ".join(generate(s) for s in rule)
print("Quest:", generate("S"))
```

You hear someone counting money.
You stop. Hachi is in the way!
You displaced Hachi.
You displaced Hachi.

Games to study

- Speedtree (https://en.wikipedia.org/wiki/SpeedTree)
  Acquired by Unity
- Nethack (https://en.wikipedia.org/wiki/NetHack)
  Automatic generation of rooms
- Diablo (https://en.wikipedia.org/wiki/Diablo)
  Loot systems
- Dwarf Fortress (https://en.wikipedia.org/wiki/Dwarf_Fortress)
  Grammars for names and world's history
- Caves of Qud (https://en.wikipedia.org/wiki/Caves_of_Qud)
  Extended grammars to generate meaning to the generation. The map is static (same for everyone, but the history is tailored to the individual).

Boby6kille the Hatamoto St:16 Dx:17 Co:18 In:9 Wi:9 Ch:6 Lawful
Dlvl:2 $:130 HP:15(15) Pw:2(2) AC:4 Exp:1 T:668 Burdened